Feedback — kwestionariusz wyboru silnika

Status: silnik nie został wybrany. Materiał do dyskusji zespołu, 19.10.2026. Porównujemy Unity, Bevy, Unreal Engine i Godot bez ustalonego zwycięzcy. Kolejność nie oznacza rankingu.

Co wiemy, a co trzeba potwierdzić

Potwierdzone przez zespół: dobra znajomość C# i Rust; część osób pracowała w Unreal Engine; część tworzyła proste gry w Bevy na innym przedmiocie około rok temu. Dostępny jest eksport mockupu Figma Make.

Zapisane w dokumentacji, do potwierdzenia aktualności: platformówka 2D z parowaniem, transferem pędu i zagadkami; 3 poziomy i 2 typy przeciwników; Windows 10/11; 60 FPS na grafice zintegrowanej; restart do 1 s; budżet pieniężny 0 zł; deterministyczne odtwarzanie wejść. Starsze materiały używają też etykiety roguelike. Nie zakładamy, że wszystkie te zapisy nadal obowiązują.

Nieustalone: liczba osób samodzielnych w każdym silniku, zakres ich doświadczenia, docelowa oprawa, sposób edycji poziomów, aktualny budżet czasu i wymagania prowadzącego na jutro.

Jak wypełnić

Propozycja organizacyjna: każdy najpierw uzupełnia tabelę doświadczenia; potem zespół wspólnie odpowiada na pytania. Można wpisać „nie wiemy”. Nie przyznajemy punktów za samą popularność silnika ani znajomość jego języka. Oddzielamy preferencję od doświadczenia i wyników próby.

1. Kto potrafi co zrobić?

Skala: **0** — bez doświadczenia; **1** — ćwiczenia/tutorial; **2** — samodzielna mała gra; **3** — projekt z buildem dla innych i rozwiązywaniem problemów. To skala doświadczenia, nie ranking silników. Opis projektu jest ważniejszy od liczby.

Osoba	C#	Rust	Unity 0–3	Bevy 0–3 + wersja	Unreal 0–3 + BP/C++	Godot 0–3 + język	Co dowieziono / link
…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…

Doprecyzowanie doświadczenia Bevy: czy wcześniejsze gry obejmowały kolizje, UI, animacje, poziomy w plikach i eksport na Windows? Które biblioteki wykorzystano? Czy da się uruchomić stare repozytorium? Nie zakładamy konieczności migracji — dopiero ustalamy, czy użyć starej, przypiętej wersji, czy nowszej.

Doprecyzowanie doświadczenia Unreal: czy chodzi o Blueprinty, C++, projekt 2D/Paper 2D czy 3D? Czy ktoś przygotował samodzielny build? Doświadczenie w UE jest atutem, ale zakres ma znaczenie.

Odpowiedzi / osoby mogące pomóc innym: …

2. Co rzeczywiście ma być grą?

Wybierzcie lub opiszcie: **platformówka boczna 2D / widok z góry / izometria / 2.5D / jeszcze nieustalone**. Czy poziomy będą ręcznie projektowane, proceduralne czy mieszane? Czy mockup odzwierciedla docelowy produkt, czy wyłącznie kierunek wizualny?

Znaczenie: wszystkie cztery silniki obsłużą mechanikę parowania. Istotna rola scen 3D, kamery i oświetlenia zwiększa znaczenie doświadczenia w Unreal; płaska grafika i tilemapy kierują uwagę na workflow 2D. Żaden wariant sam nie rozstrzyga wyboru.

Odpowiedź: …

3. Co jest obowiązkowe na ocenę i na jutro?

Co dokładnie jest wymagane: **pitch i plan technologii / pokaz makiety / grywalny prototyp / inne**? Czy prowadzący wymaga konkretnego silnika, funkcji, platformy lub uzasadnienia wyboru? Czy jutro można uczciwie pokazać short-listę i plan próby zamiast deklarować decyzję?

Znaczenie: zakres prezentacji i zakres końcowej gry to osobne ustalenia. Nie opisujemy planowanych funkcji jako już działających.

Odpowiedź: ...

4. Co chcemy osiągnąć poza zaliczeniem?

Ustalcie własną kolejność: **dopracowana gra / nauka silnika / rozwój Rust lub C# / portfolio / wspólna praca / eksperyment techniczny**. Jaką część dostępnego czasu akceptujemy na naukę i narzędzia?

Znaczenie: nauka Bevy lub Unreal może być celem sama w sobie. Jeśli liczy się głównie gotowa gra, przewagę daje rzeczywiście opanowany workflow. Nie zakładamy, że mniej kodu oznacza dla tego zespołu mniej pracy.

Priorytety i limit czasu: ...

5. Kto będzie tworzył poziomy i balansował grę?

Czy autorzy poziomów chcą pracować w edytorze wizualnym? Czy powinni zmieniać układ mapy, parametry przeciwników i okno parry bez zmiany kodu i rekompilacji? Kto zrobi trzy poziomy, a kto przejmie tę pracę podczas nieobecności autora?

Znaczenie: Unity, Godot i Unreal oferują zintegrowane edytory. Przy Bevy należy wskazać konkretny workflow: np. LDtk + kompatybilny importer, własne dane lub wybrany edytor. To nie oznacza konieczności pisania edytora od zera. Aktualny projekt edytora Bevy trzeba ocenić na używanej wersji, bez traktowania roadmapy jako gotowej funkcji. [B2, B4]

Autor / zastępca / narzędzie / format: ...

6. Jak ma działać ruch i parowanie?

Czy potrzebujemy pełnej symulacji brył, czy kontrolowanego ruchu postaci i pocisków z wybranymi reakcjami na kolizje? Czy parry odwraca pocisk, odbija go według kąta, czy pozwala celować? Co dzieje się z grawitacją i sterowaniem po impulsie?

Znaczenie: silnik nie dostarcza automatycznie satysfakcjonującego parry. W każdym wariancie potrzebne są reguły ruchu, hitboxy i timing. W Bevy trzeba też wskazać warstwę kolizji/fizyki, np. Avian, Rapier albo własne testy kolizji dla ograniczonego zakresu. Wersje pluginów muszą pasować do silnika. [B5, B6]

Reguły / niewiadome do sprawdzenia: ...

7. Co oznacza „deterministyczna” gra w wymaganiach?

Czy chodzi o podobne zachowanie przy różnych FPS, powtarzalne wyniki na tym samym buildzie i komputerze, czy identyczny stan między komputerami? Czy porównujemy wynik poziomu, pozycje z tolerancją czy dokładne wartości? Czy wymóg jest obowiązkowy?

Znaczenie: stały krok symulacji nie jest dowodem determinizmu. Rust ani ECS też go nie gwarantują. Godot wprost ostrzega o niedeterministyczności fizyki; w pozostałych wariantach trzeba sprawdzić konkretny backend, konfigurację i zakres gwarancji. To pytanie może zmienić architekturę niezależnie od silnika. [G2]

Zakres i test akceptacyjny: ...

8. Na czym tworzymy i na czym pokazujemy grę?

Jakie systemy, CPU, RAM i GPU ma zespół? Jaki komputer będzie na prezentacji? Czy nadal celujemy w Windows, czy potrzebny jest również Linux lub link do wersji WWW? Czy w sali będzie dostęp do sieci?

Znaczenie: trzeba sprawdzić działający eksport i sterowniki na konkretnym sprzęcie. Nie zakładamy automatycznie, że Bevy będzie szybsze, Unreal za ciężki, a Godot uruchomi się wszędzie. C# w Godot 4 ma obecnie ograniczenie eksportu WWW; staje się to istotne dopiero, gdy potwierdzimy potrzebę WWW. [G3]

Sprzęt / platformy / sposób dostarczenia: ...

9. Jak szybka musi być iteracja?

Ile czasu akceptujemy od zmiany parametru lub skryptu do sprawdzenia go w grze? Czy przy każdej zmianie potrzebny jest build, czy konfiguracja może być przeładowywana? Czy mierzymy to na komputerze autora poziomów?

Znaczenie: w Bevy należy zmierzyć kompilację przyrostową i sposób ładowania danych; oficjalne materiały opisują sposoby skracania kompilacji. W pozostałych silnikach również mierzymy faktyczną pętlę pracy, w tym import zasobów i przebudowę kodu. [B3]

Akceptowalny czas / pomiar: ...

10. Kto utrzyma projekt, gdy główna osoba wypadnie?

Kto poza autorem uruchomi projekt, naprawi błąd parry i przygotuje build? Czy jesteśmy gotowi przypiąć wersje silnika, pluginów i toolchainu na czas projektu? Co robimy, gdy potrzebna biblioteka nie wspiera wybranej wersji?

Znaczenie: znajomość silnika przez jedną osobę może przyspieszyć start, ale stworzyć wąskie gardło. Bevy dokumentuje częste zmiany API; przypięcie wersji ogranicza koszt migracji, lecz nadal trzeba sprawdzić zestaw pluginów. Podobną dyscyplinę stosujemy do wersji Unity, Unreal i Godot. [B1]

Osoba główna / zastępca / polityka wersji: ...

11. Jakie ograniczenia finansowe i licencyjne obowiązują?

Czy 0 zł nadal jest warunkiem? Czy gra pozostanie niekomercyjna? Czy zespół dopuszcza płatne pluginy lub assety? Czy są ograniczenia uczelni dotyczące kont i licencji?

Znaczenie: Godot i Bevy mają licencje permissive. Unity Personal i Unreal mogą pasować do projektu, ale trzeba sprawdzić warunki konkretnego użycia. Osobno sprawdzamy licencje grafik, fontów, muzyki i zależności. Nie zakładamy planu sprzedaży. [L1–L4]

Odpowiedź / ograniczenia: ...

12. Co przesądzi o decyzji?

Dla każdego silnika wpiszcie **największy atut dla naszego zespołu**, **największą obawę** oraz **jedną próbę, która ją rozstrzygnie**. Czy mamy dość danych, żeby wybrać bez porównywania wszystkich czterech implementacji?

Silnik	Nasz konkretny atut	Obawa	Dowód / próba	Osoba odpowiedzialna
Unity	...	...	...	...
Bevy	...	...	...	...
Unreal	...	...	...	...
Godot	...	...	...	...

Karty silników — korzyści i koszty do skonfrontowania z odpowiedziami

Unity — C# i zintegrowany workflow

Możliwe korzyści: wykorzystanie znanego C#; narzędzia do sprite'ów, tilemap, fizyki 2D i animacji; wizualna praca nad poziomami oraz UI. Zmniejsza potrzebę dobierania podstawowych narzędzi z różnych projektów. [U1]

Koszty / pytania: znajomość C# nie oznacza jeszcze znajomości komponentów, cyklu życia Unity i pracy z zasobami. Trzeba ustalić pakiety, wersję edytora, pracę nad wspólnymi scenami i warunki licencji. Nie ma tu potwierdzenia wcześniejszych projektów zespołu w Unity.

Próba rozstrzygająca: osoba znająca C# tworzy ruch, parry i małą scenę; druga osoba zmienia układ i balans bez pomocy autora. Mierzymy czas oraz trudności.

Bevy — Rust i wcześniejsze małe gry zespołu

Możliwe korzyści: bezpośrednie wykorzystanie Rust oraz rzeczywistego doświadczenia w Bevy; ECS może pasować do komponentów pocisków, parry, przeciwników i przełączników; kod oraz dane mogą dobrze współgrać z przeglądami zmian. Silnik obsługuje 2D i 3D i jest rozwijany jako zestaw modułów. [B1]

Koszty / pytania: trzeba wybrać i sprawdzić workflow poziomów, UI, kolizji oraz zgodne pluginy. Oficjalny edytor jest rozwijany; nie utożsamiamy prototypów z dojrzałym workflow Unity/UE/Godot. Dochodzą czasy kompilacji oraz zmiany API. Doświadczenie sprzed roku należy powiązać z konkretną wersją, a nie traktować jako nieaktualne z definicji. [B2, B3]

Próba rozstrzygająca: uruchomić wcześniejszą grę, wskazać przydatne wzorce i przygotować minimalny zestaw wersji: silnik + kolizje + edycja poziomu. Osoba inna niż autor ma zmienić mapę i wyeksportować build. Nie zakładamy, że stary kod jest odpowiedni ani że wymaga migracji.

Unreal Engine — doświadczenie części zespołu i rozbudowany edytor

Możliwe korzyści: znany części zespołu workflow; Blueprints umożliwiają wizualne definiowanie logiki, a C++ rozszerzanie kodu. Paper 2D oferuje sprite'y, flipbooki i tilemapy; możliwa jest też oprawa 2.5D/3D. Gra 2D nie wyklucza Unreal. [E1]

Koszty / pytania: doświadczenie w grze 3D nie rozstrzyga znajomości Paper 2D. Potrzebne jest sprawdzenie ruchu na płaszczyźnie, kolizji, organizacji Blueprintów, pracy na zasobach i eksportu na docelowy sprzęt. Znajomość C# i Rust nie przenosi się bezpośrednio na C++/Blueprinty. Rozbudowane narzędzia pomagają tylko wtedy, gdy zespół umie z nich korzystać.

Próba rozstrzygająca: osoba z doświadczeniem UE robi scenę 2D z parry i samodzielny build, a druga osoba wprowadza zmianę. Sprawdzamy czas pracy oraz wydajność, zamiast odrzucać UE za samą etykietę „silnik 3D".

Godot — narzędzia 2D i możliwość użycia C#

Możliwe korzyści: dedykowane węzły 2D, sceny, tilemapy i kontroler CharacterBody2D; C# pozwala wykorzystać znany język bez konieczności wybierania GDScript. Zewnętrzne zasoby umożliwiają oddzielenie balansu od kodu. [G1, G4]

Koszty / pytania: nie potwierdzono wcześniejszej pracy zespołu w Godot. Trzeba nauczyć się modelu scen/węzłów i sprawdzić workflow .NET, eksport oraz integrację zasobów. Użycie C# nie eliminuje nauki silnika. Eksport WWW w Godot 4 C# jest obecnie ograniczeniem, jeśli WWW okaże się wymaganiem. Fizyka nie daje automatycznej gwarancji determinizmu. [G2, G3]

Próba rozstrzygająca: mała scena w C# lub innym świadomie wybranym języku; druga osoba zmienia parametry i poziom w edytorze, a build działa na docelowym Windows.

Wspólna próba — jeśli po rozmowie zostanie dwóch kandydatów

To propozycja do zatwierdzenia, nie ustalony zakres implementacji. Ustalcie jednakowy limit czasu i podobną scenę dla obu. Nie porównujcie samego FPS ani jakości gotowych assetów. Zapisujcie, kto wykonywał próbę i jakie miał doświadczenie.

Scena: ruch, skok, parry, jeden przeciwnik z pociskiem, przełącznik i drzwi; osobno sparowany impuls ruchu; śmierć i reset. Parametry poza kodem. Potem osoba inna niż autor zmienia poziom i parametr parry.

Co sprawdzamy	Unity	Bevy	Unreal	Godot
Czas do działającej sceny	...	...	...	...
Czas zmiany parametru → sprawdzenie w grze	...	...	...	...
Autor poziomu pracuje samodzielnie	...	...	...	...
Okno parry niezależne od FPS renderowania	...	...	...	...
Kolizje przy największej planowanej prędkości	...	...	...	...
Powtarzalność według odpowiedzi na pytanie 7	...	...	...	...
FPS i czas resetu na ustalonym komputerze	...	...	...	...
Build działa na drugim komputerze	...	...	...	...
Druga osoba potrafi naprawić prosty błąd	...	...	...	...
Dodatkowy kod, pluginy i nierozwiązane problemy	...	...	...	...

Arkusz decyzji — pozostaje pusty do rozmowy

- Potwierdzony zakres gry i obowiązkowe platformy: ...
- Trzy najważniejsze kryteria zespołu: ...
- Kandydaci pozostający po rozmowie: ...
- Wymaganie, którego kandydat nie spełnia, i dowód: ...
- Próba potrzebna przed wyborem / osoba / termin: ...
- Decyzja: **otwarta / warunkowa / potwierdzona**: ...
- Wybrany silnik i wersja, jeśli ustalono: ...
- Dlaczego pasuje do tego zespołu i zakresu: ...
- Koszt lub ograniczenie, które świadomie akceptujemy: ...
- Właściciel projektu i zastępca: ...
- Co można powiedzieć jutro o technologii bez wyprzedzania decyzji: ...

Źródła

Fakty o funkcjach i licencjach pochodzą ze źródeł producentów i projektów; przewidywany wpływ na pracę zespołu jest oceną do weryfikacji. Wersje oraz zgodność dodatków należy przypiąć dopiero przy wyborze konkretnego zestawu. Weryfikacja stron: 19.10.2026.

- B1: Bevy — Introduction: Rust, ECS, licencja i rozwój API.
- B2: Bevy — Sixth Birthday, stan scen, UI i prac nad edytorem, 10.08.2026.
- B3: Bevy — Setup, konfiguracja i czasy kompilacji.
- B4: bevy_ecs_ldtk — dokumentacja importera i zgodność wersji.
- B5: Avian 2D — dokumentacja projektu.
- B6: Rapier — integracja z Bevy.
- U1: Unity — narzędzia 2D.
- E1: Epic — Paper 2D, opis systemu; dokumentacja wersji 5.2. Nie jest to potwierdzenie zgodności pluginów z dowolną nowszą wersją.
- G1: Godot — CharacterBody2D.
- G2: Godot — Physics introduction, ograniczenia determinizmu.
- G3: Godot — eksport WWW i ograniczenia C#.
- G4: Godot — Resources, dane poza logiką.
- L1: Godot — MIT.
- L2: Bevy — repozytorium i licencje MIT / Apache 2.0.
- L3: Unity — plany i kwalifikacja, anulowanie Runtime Fee.
- L4: Unreal Engine — warunki licencyjne.